CRM DE ATENDIMENTO E ORÇAMENTOS

Entendendo o Backend

Parte 5 — Autenticação e usuários

Quem pode entrar, o que cada um pode fazer — e como as senhas ficam protegidas

1 2 3 4 5

Parte 5 de 5 — a última do backend

VISÃO GERAL

Onde estamos no plano do backend

1

Fundação + Domínio

Regras do negócio em Python puro, com testes.

✓ CONCLUÍDA

2

Casos de uso

As ações do usuário, orquestrando o domínio.

✓ CONCLUÍDA

3

Persistência

Banco de dados, tabelas e migrations.

✓ CONCLUÍDA

4

API REST

As rotas HTTP que o frontend chama.

✓ CONCLUÍDA

5

Autenticação

Login, token, senhas e gestão de usuários.

VOCÊ ESTÁ AQUI

Até a Parte 4, qualquer pessoa que soubesse o endereço da API podia mexer no funil. A Parte 5 fecha a porta: só entra quem tem login.

Com ela, o backend está completo: todas as rotas da especificação, todos os RFs do backend e os RNFs de segurança e desempenho.

CONCEITO

Duas perguntas diferentes

Autenticação

Quem é você?

O login confere e-mail e senha e entrega um token. Sem token válido, a API nem conversa.

401 Não autenticado: entre no sistema primeiro.

Autorização

Você pode fazer isso?

Já sabemos quem é. Mas só o Administrador gerencia usuários; o atendente comum usa o funil.

403 Acesso negado: perfil insuficiente.

Analogia: o crachá na portaria é a autenticação (você entra no prédio). A chave da sala do diretor é a autorização (nem todo crachá abre).

COMO FUNCIONA

O caminho do login, passo a passo

1

Frontend envia

POST /auth/login
e-mail + senha

2

API confere

busca o usuário e compara a senha com o hash

3

API devolve o token

válido por 8 horas

4

Frontend guarda

e envia em cada pedido: Authorization: Bearer ...

5

API confere o token

e recarrega o usuário do banco a cada pedido

Por que o token?

A senha é enviada uma única vez, no login. Depois disso só circula o token — que expira sozinho em 8 horas, um turno de trabalho.

Por que recarregar o usuário?

Se o administrador desativar alguém, o acesso cai no próximo pedido — sem esperar o token dessa pessoa expirar.

RNF07

Senhas nunca ficam guardadas como texto



O que o banco guarda é só a coluna da direita. Do hash não dá para voltar à senha.

1 Sal aleatório

Dois usuários com a mesma senha ficam com hashes diferentes.

2 Lento de propósito

Cada tentativa custa ~60 ms: adivinhar milhões de senhas fica caro.

3 Custo gravado junto

Os números 16384$8$1 permitem aumentar o custo no futuro.

Por que scrypt e não bcrypt? Os dois são próprios para senhas, mas o scrypt já vem no Python (hashlib). bcrypt e argon2 trazem DLLs compiladas — que o Smart App Control desta máquina bloquearia, como bloqueou o lxml e o SQLAlchemy.

POR DENTRO DO TOKEN

JWT: um crachá que ninguém consegue falsificar

Um token tem três partes, separadas por ponto:

eyJhbGciOiJIUzI1NiJ9	eyJzdWIiOiIwOWJl...	SflKxwRJSMeKKF2Q...
cabeçalho	**conteúdo**	**assinatura**
o algoritmo (HS256)	quem é e até quando vale	o lacre, feito com JWT_SECRET

O conteúdo, decodificado:

```
{
  "sub": "09be6200-d507-...",  # id do usuário
  "perfil": "ADMINISTRADOR",
  "iat": 1760221547,           # emitido em
  "exp": 1760250347            # expira em
}
```

Qualquer um lê, ninguém falsifica

O conteúdo não é secreto — por isso a senha nunca vai nele. Mas alterar uma vírgula invalida a assinatura, e sem o JWT_SECRET ninguém gera uma assinatura nova.

A validade é conferida pelo relógio da aplicação — assim os testes conseguem “avançar 8 horas” e ver o token expirar.

NA PRÁTICA

Como cada rota do CRM fica protegida

```
# main.py — todas as rotas do CRM exigem login
exige_login = [Depends(obter_usuario_logado)]
app.include_router(orcamento_controller.router,
                   dependencies=exige_login)

# dependencias.py
def obter_usuario_logado(credenciais, emissor, usuarios):
    if credenciais is None:
        raise NaoAutenticado()
    usuario_id = emissor.ler(credenciais.credentials)
    usuario = usuarios.buscar_por_id(usuario_id)
    if usuario is None or not usuario.ativo:
        raise NaoAutenticado()
    return usuario
```

O porteiro confere, em ordem:

Sem token?	**401**
Token falso ou expirado?	**401**
Usuário apagado ou desativado?	**401**
Tudo certo	**a rota roda**

Só o login e a documentação (/docs) ficam abertos.

UM CUIDADO DE SEGURANÇA

Login errado: sempre a mesma resposta

Tentativa	Resposta da API
Senha errada	HTTP 401
E-mail que não existe	{
E-mail mal formado	"erro": "CredenciaisInvalidas",
Usuário desativado	"mensagem": "E-mail ou senha incorretos.",
	"campo": null
	}

Por que não dizer “e-mail não cadastrado”?

Porque isso ensinaria a um invasor quais e-mails existem no sistema — meio caminho para adivinhar a senha.

E o tempo de resposta também é igual

Mesmo com e-mail inexistente, a API calcula um hash de referência — senão a resposta rápida denunciaria o e-mail inexistente.

UC10 · RF10

Gestão de usuários: só o Administrador

GET	/usuarios	lista todos
POST	/usuarios	cria (senha ≥ 8 caracteres)
PATCH	/usuarios/{id}	edita nome, e-mail, perfil, ativo

Respostas de erro:

403	Usuário comum tentando gerenciar
409	E-mail que já pertence a outro usuário
422	Senha curta, ou administrador se desativando
404	Usuário que não existe

A checagem de perfil fica no caso de uso, não só na rota

```
def _exigir_administrador(solicitante):
    if not isinstance(solicitante, Administrador) or not solicitante.ativo:
        raise AcessoNegado()
```

Mesmo que alguém chame o caso de uso por outro caminho (um script, um teste), o usuário comum é barrado.

O sistema nunca fica sem administrador

```
class Administrador(Usuario):

    def desativar_usuario(self, usuario):
        if usuario.id == self.id:
            raise OperacaoNaoPermitida(
                "Um administrador não pode desativar a própria conta.")
        ...

    def definir_perfil(self, usuario, perfil):
        if usuario.id == self.id:
            raise OperacaoNaoPermitida(
                "Um administrador não pode alterar o próprio perfil.")
        classe = Administrador if perfil is ADMINISTRADOR else Usuario
        return classe(id=usuario.id, nome=..., email=..., ...)
```

1. **Sem auto-desativação**
   Exigência do RF10.

2. **Sem auto-rebaixamento**
   Mesmo motivo: o último administrador poderia se trancar fora.

3. **Promover é recriar**
   Trocar de perfil devolve um objeto da outra classe, com o mesmo id.

Como são regras de negócio, ficam na entidade da Parte 1 — e valem para qualquer caminho, não só para a API.

O OVO E A GALINHA

Quem cria o primeiro administrador?

Só um administrador cria usuários — mas, num banco novo, não existe nenhum. Para isso há um script que roda direto no servidor:

```
cd backend
.venv\Scripts\python.exe src\criar_administrador.py --nome "Dona Maria" --email maria@empresa.com
Senha (mínimo 8 caracteres):
Repita a senha:
Administrador criado: Dona Maria <maria@empresa.com>
```

1 Só com o cadastro vazio

Depois do primeiro, o script recusa: novos usuários só pela API.

2 Senha escondida

Pedida no terminal sem aparecer na tela, e digitada duas vezes.

3 Sem rota pública

Não existe “criar conta” aberto na API.

RNF01

O painel com 500 orçamentos

limite da especificação

2.000 ms

para carregar o Kanban

medido nesta máquina

~80 ms

cerca de 25 vezes mais rápido

Como o teste funciona

- Cria 500 clientes, cada um com um orçamento e 3 interações (1.500 no total).
- Mede o GET /orcamentos e falha se passar de 2 segundos.
- Mesmo com o SQLAlchemy em Python puro (por causa do Smart App Control) e SQLite, sobra folga. Ajudam: a projeção enxuta do card e o carregamento das interações em uma consulta só.

RESULTADO

Como a Parte 5 — e o backend — terminaram

300

testes passando

90 pulados: variantes PostgreSQL

5 de 5

partes concluídas

o backend inteiro

~80 ms

painel com 500 orçamentos

limite: 2.000 ms

0

senhas em texto puro

só hash scrypt com sal

Também testado com o servidor de verdade: primeiro administrador criado, 401 sem token, 401 com senha errada, login, painel aberto com o token e criação de uma atendente.

MÃO NA MASSA

Entre no sistema pelo navegador

```
cd backend
set DATABASE_URL=sqlite:///./crm_dev.db
.venv\Scripts\python.exe -m alembic upgrade head
.venv\Scripts\python.exe src\criar_administrador.py --nome "Eu" --email eu@crm.com
.venv\Scripts\python.exe -m uvicorn main:app --app-dir src --reload
```

Exercício: autenticação × autorização na prática

1. Em http://localhost:8000/docs, chame GET /orcamentos sem login: veja o 401.
2. Faça POST /auth/login, copie o accessToken e cole no botão Authorize (cadeado).
3. Crie um usuário comum em POST /usuarios. Saia (Logout), entre com ele e tente GET /usuarios: veja o 403.
4. Volte como administrador e desative-o com PATCH /usuarios/{id} {"ativo": false}: o token dele para de funcionar na hora.

PARA REVISAR

Glossário rápido e próximo passo

Autenticação	Provar quem você é (login). Falhou: 401.
Autorização	Ter permissão para a ação (perfil). Falhou: 403.
Hash de senha	Resumo irreversível da senha; o banco só guarda ele.
Sal	Valor aleatório que deixa hashes iguais diferentes.
JWT	Token assinado com quem é o usuário e até quando vale.
Bearer	Jeito de enviar o token: Authorization: Bearer ...

PRÓXIMOS PASSOS

Backend completo

- Validar tudo no PostgreSQL real (Python 64 bits + Docker).
- Decidir a rota para abrir um 2º orçamento de um cliente (RF08).
- Fazer o commit das Partes 2 a 5.
- Depois: o frontend em Next.js — o Kanban que vai consumir esta API.